Flakiness Repair in the Era of Large Language Models

Yang Chen
University of Illinois Urbana-Champaign, USA
yangc9@illinois.edu

ABSTRACT

Flaky tests can non-deterministically pass or fail regardless of any change to the code, which negatively impacts the effectiveness of the regression testing. Prior repair techniques for flaky tests mainly leverage program analysis techniques to mitigate test flakiness, which only focus on Order-Dependent (OD) and Implementation-Dependent (ID) flakiness with known flakiness patterns and root causes. In this paper, we propose an approach to repair flaky tests with the power of Large Language Models (LLMs). Our approach successfully repaired 79% of OD tests and 58% of ID tests in an extensive evaluation using 666 flaky tests from 222 projects. We submitted pull requests to fix 61 flaky tests; at the time of submission, 19 tests have already been accepted. However, we observed that currently LLMs are ineffective in adequately repairing Non-Order-Dependent (NOD) flaky tests by analyzing 118 of such tests from 11 projects.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**.

KEYWORDS

Software Testing, Test Flakiness, Large Language Models

1 RESEARCH PROBLEM

Test flakiness occurs when non-deterministic behavior in test execution results is observed without any changes in the code under test, significantly impacting software quality. Common flaky tests can be categorized into two categories: OD and NOD tests, depending on whether test results depend on the test order in which the tests are run [12]. ID tests [3] represent a specific type of NOD flakiness, which can happen due to wrong assumptions of unordered collections in the implementation. Several research projects have been proposed for characterizing [6, 8, 10, 11, 13, 14] or detecting [5, 9, 12, 17, 20, 21, 23–26, 28] flaky tests. State-of-the-art techniques for repairing flaky tests include three groups: (1) iFixFlakies [22] and iPFlakies [24] focus on repairing Java and Python OD tests, respectively. They first take the OD tests along with the failing test order and the passing test order, and then find cleaner tests that can modify the shared state, finally use cleaners to generate patches. Therefore these techniques rely on the existence of cleaner tests. ODRepair [15] is proposed to overcome such limitation; it analyzes the static fields and serialized heap state to identify polluted shared states between Java tests, and relies on test generation techniques to generate cleaner statements to resolve the flakiness. (2) DexFix [27] is designed to repair ID flakiness by implementing four types of specific strategies (e.g., replacing `HashSet` with `LinkedHashSet`) for ID tests. (3) TRaF [19] aims to address NOD flaky tests caused by asynchronous waits in JavaScript tests by updating the waiting time of asynchronous calls.

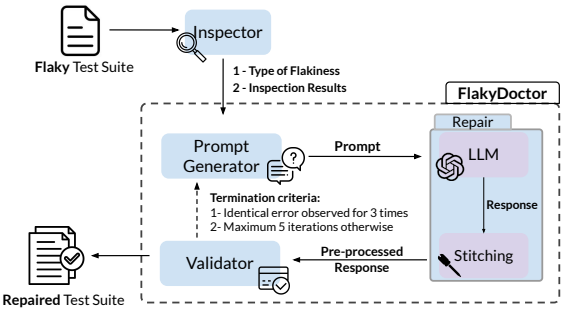


Figure 1: Overview of FlakyDoctor for repairing flaky tests

To overcome the generalizability limitations, LLMs can be applied to flakiness repair. In this paper, we propose an approach that integrates the power of LLMs with the combination of program analysis to repair flaky tests. We specifically focus on OD and ID tests, as reliable detection of NOD tests is still lacking, making it challenging to guarantee the validation of patches for NOD flakiness. To our knowledge, our approach is the first technique for repairing multiple categories of flaky tests using the capabilities of LLMs. Compared to iFixFlakies and ODRepair, our approach does not rely on the existence of cleaner tests and can repair both OD-Victim and OD-Brittle tests. Compared to DexFix, our approach is not limited to the specific patterns of ID flakiness.

2 APPROACH

In this section, we will discuss our approach and the effectiveness of each component. Figure 1 shows the overview of FlakyDoctor, which contains *four* phases: *Inspector*, *Prompt Generator*, *Repair*, and *Validator*. When provided with the original flaky test suite and the flakiness identified by detection tools, the *Inspector* component first examines and reproduces the test flakiness, producing two outputs for the *Prompt Generator* component: (1) test execution results and (2) location of test failures. *Prompt Generator* processes the above two messages and generates a prompt with *five* sections: *Instruction*, *Problem Definition*, *Related Code*, *Failure Location*, and *Rules*. *Prompt Generator* tailors the problem definition according to different types of flakiness. Instead of providing the entire failure message to the model, *Prompt Generator* analyzes the stack trace of the test failure. It identifies the specific statements within the test code that involve the test failures and only provide these statements in the prompt. To show the effectiveness of the prompt crafting, we evaluated the same LLM for *vanilla prompting* without additional context, in

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSE-Companion '24, April 14–20, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0502-1/24/04…$15.00
https://doi.org/10.1145/3639478.3641227

```
1  @Test // ID flaky test
2  public void testPriority() throws Exception {
3    //...
4    String cvs1 = (String) mbs.getAttribute(mxbeanName,
5    "CallVolumeSummary");
6  - assertTrue("Get expected JMX of CallVolumeSummary before decay",
7  -       cvs1.equals("{\"A\":6,\"B\":2,\"C\":2}"));
8  + Map<String, Integer> map1 =
9  + new Gson().fromJson(cvs1, new TypeToken<LinkedHashMap<String,
10 +    Integer>>().getType());
11 + Map<String, Integer> expectedMap1 = new LinkedHashMap<>();
12 + expectedMap1.put("A", 6);
13 + expectedMap1.put("B", 2);
14 + expectedMap1.put("C", 2);
15 + assertEquals("Get expected JMX of CallVolumeSummary before decay",
16 +     expectedMap1, map1);
17 }
```

Figure 2: A Real-world ID flaky test from Hadoop [4]

contrast to the effective results achieved by FlakyDoctor, the LLM was able to generate reasonable response (with potential patches included) for 34.9% tests, and repair only 2.6% of all the flaky tests.

Table 1: Effectiveness of (1) FlakyDoctor and DexFix in repairing ID flakiness, (2) FlakyDoctor in repairing NOD flakiness, and (3) FlakyDoctor, ODRepair and iFixFlakies in repairing OD flakiness. PF: Previously Fixed tests; PU: Previously Unfixed tests; FP: False Positive (excluded from PF and PU).

Category	#Projects	#Tests		#FlakyDoctor			DexFix Dataset (ID)	
		PF	PU	PF	PU	FP	#FlakyDoctor	#DexFix
ID	195	314	105	198	47	13	86	72
NOD	11	5	113	0	2	6	-	-
Category	#Projects	#Tests		#FlakyDoctor			#ODRepair	#iFixFlakies
		PF	PU	PF	PU	FP		
OD-Victim	35	187	27	156	12	3	123	92
OD-Brittle	16	28	5	16	1	2	-	-

Once the prompts are generated, they are sent to the LLMs in *Repair* component. In our approach, we use GPT-4 [1] as the LLM because (1) GPT-4 has outperformed other language models in several research projects [7, 16, 18] and (2) GPT-4 is more accessible with the longest context window. When the LLM responds to the prompt with a patch, *Repair* component initiates several checks for simple compilation errors (e.g., class dependency checks) and attempts to resolve them based on some basic heuristics (e.g., add missing class dependencies). The modified patch then undergoes the *Validator* component to check if it can resolve the flakiness. If validated, the patch is deemed ultimately successful. Otherwise, the patch generated from the current response will proceed into *Prompt Generator* with the latest test execution outputs. A new prompt updated with the latest information will start another iteration of repair. The iterative repair process continues until a successful patch is generated or after up to five iterations. Of all the tests that are repaired, 51% are addressed in the first iteration, and subsequent iterations result in about doubling the number of fixed tests.

Figure 2 shows a patch generated by FlakyDoctor for an ID flaky test `testPriority` from Hadoop [4]. The flakiness is due to converting a `Json` object into `String` (Line 4). In Line 6, the assertion checks if the conversion equals to {"A":6,"B":2,"C":2}, assuming the elements in the conversion of the `Json` object are always in the same order. As a result, this test non-deterministically passes or fails. To repair this test, the patch generated by FlakyDoctor first transforms the converted `String` (`csv1`) into an ordered collection `LinkedHashMap` (Lines 8–10), and then reconstructs the expected output in the previous assertion as a `LinkedHashMap` (Lines 11–14), finally comparing these two objects in the new assertion (Line 15) resolves the flakiness.

3 RESULTS

To collect subjects for ID tests, we use IDoFT [2], which contains more than 5490 Java flaky tests of different types detected in real-world projects. We excluded projects that (1) were deleted, (2) were not able to compile in a reasonable time due to non-trivial issues such as deprecated dependencies, and (3) were no longer able to reproduce flakiness. Starting from the modules of the remaining projects from the previous step, we sampled one already fixed flaky test by previous techniques (so we can compare FlakyDoctor with prior work) and one unfixed flaky test (so we can evaluate FlakyDoctor on real cases where we can open pull requests) if they exist. We also included the dataset of DexFix. Finally, our collected dataset consists of a total of 419 ID tests from 195 projects and 118 NOD tests from 11 projects. For OD tests, we used the ODRepair dataset and excluded projects that were not available, similar to the steps for filtering ID tests. This resulted in a total of 214 OD-Victim tests from 35 projects. We also include all the 33 OD-Brittle tests from the IDoFT dataset.

Table 1 shows the projects utilized in our experiments and the corresponding evaluation results for ID, NOD, and OD tests. From the results, we can see that FlakyDoctor can repair 58% ID tests. Breaking down the results, it was able to repair 63% of previously fixed and 45% of previously unfixed ID flaky tests. When comparing our approach with DexFix, FlakyDoctor was able to repair 55% of ID tests in the DexFix Dataset, while the success rate of DexFix is 46%. Regarding the OD tests, our approach can successfully repair 51% of OD-Brittle tests, including one previously unfixed. For the OD-Victim tests, we achieved a higher repair success rate of 78%, while ODRepair was able to repair 57% of the total OD-Victim tests, and iFixFlakies was able to repair 43% of them.

Although FlakyDoctor is effective in repairing ID and OD flaky tests, repairing NOD flakiness remains challenging. FlakyDoctor only successfully repairs two NOD tests out of 118 tests. There can be several reasons. (1) Reproducing NOD test failures is not guaranteed. Some NOD tests exhibit flakiness only after rerunning thousands of times, which may not be allowed due to resource limitations. (2) It is challenging for LLMs to locate the actual root causes of NOD tests, as they are caused by factors beyond test orders, such as concurrency, deadlock, etc. Such problems are challenging to fix. (3) LLMs struggle to provide effective fixes for NOD tests, e.g., in some cases, the model can only eliminate specific assertions to ensure a test pass, thereby altering the test logic. Therefore, automatically repairing NOD flakiness is still an open problem.

4 FUTURE WORK

Currently, FlakyDoctor can repair OD and ID flaky tests using the capabilities of GPT-4. However, the ability to repair NOD flaky tests remains limited; the localization of NOD flakiness and validation of patches for them are still open problems. In future work, we aim to explore efficient methods for categorizing, detecting, and repairing NOD tests without rerunning tests with significant resources.

REFERENCES

[1] 2023. GPT-4 Technical Report. https://cdn.openai.com/papers/gpt-4.pdf.

[2] 2023. International Dataset of Flaky tests. https://github.com/TestingResearchIllinois/idoft.

[3] 2023. Nondex Test Flakiness Detection Tool. https://github.com/TestingResearchIllinois/NonDex.

[4] 2023. Repository Hadoop. https://github.com/apache/hadoop.

[5] Jonathan Bell, Owolabi Legunsen, Michael Hilton, Lamyaa Eloussi, Tifany Yung, and Darko Marinov. 2018. DeFlaker: Automatically detecting flaky tests. In *2018 IEEE/ACM 40th International Conference on Software Engineering*. IEEE, 433–444.

[6] Yang Chen, Alperen Yildiz, Darko Marinov, and Reyhaneh Jabbarvand. 2023. Transforming test suites into croissants. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1080–1092.

[7] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *arXiv preprint arXiv:2308.01861* (2023).

[8] Saikat Dutta, August Shi, Rutvik Choudhary, Zhekun Zhang, Aryaman Jain, and Sasa Misailovic. 2020. Detecting flaky tests in probabilistic and machine learning applications. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 211–224.

[9] Tariq M King, Dionny Santiago, Justin Phillips, and Peter J Clarke. 2018. Towards a Bayesian network model for predicting flaky automated tests. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion*. IEEE, 100–107.

[10] Wing Lam, Patrice Godefroid, Suman Nath, Anirudh Santhiar, and Suresh Thummalapenta. 2019. Root causing flaky tests in a large-scale industrial setting. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 101–111.

[11] Wing Lam, Kıvanç Muşlu, Hitesh Sajnani, and Suresh Thummalapenta. 2020. A study on the lifecycle of flaky tests. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 1471–1482.

[12] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A framework for detecting and partially classifying flaky tests. In *2019 12th ieee conference on software testing, validation and verification*. IEEE, 312–322.

[13] Wing Lam, Stefan Winter, Angello Astorga, Victoria Stodden, and Darko Marinov. 2020. Understanding reproducibility and characteristics of flaky tests through test reruns in Java projects. In *2020 IEEE 31st International Symposium on Software Reliability Engineering*. IEEE, 403–413.

[14] Wing Lam, Stefan Winter, Anjiang Wei, Tao Xie, Darko Marinov, and Jonathan Bell. 2020. A large-scale longitudinal study of flaky tests. *Proceedings of the ACM on Programming Languages* (2020), 1–29.

[15] Chengpeng Li, Chenguang Zhu, Wenxi Wang, and August Shi. 2022. Repairing order-dependent flaky tests via test generation. In *Proceedings of the 44th International Conference on Software Engineering*. 1881–1892.

[16] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210* (2023).

[17] Maximiliano A Mascheroni and Emanuel Irrazabal. 2018. Identifying key success factors in stopping flaky tests in automated REST service testing. *Journal of Computer Science and Technology* 18, 02 (2018), e16–e16.

[18] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2023. Understanding the Effectiveness of Large Language Models in Code Translation. *arXiv preprint arXiv:2308.03109* (2023).

[19] Yu Pei, Jeongju Sohn, Sarra Habchi, and Mike Papadakis. 2023. TRaf: Time-based Repair for Asynchronous Wait Flaky Tests in Web Testing. *arXiv preprint arXiv:2305.08592* (2023).

[20] Suzette Person and Sebastian Elbaum. 2015. Test analysis: Searching for faults in tests (N). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 149–154.

[21] Gustavo Pinto, Breno Miranda, Supun Dissanayake, Marcelo d'Amorim, Christoph Treude, and Antonia Bertolino. 2020. What is the vocabulary of flaky tests?. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 492–502.

[22] August Shi, Wing Lam, Reed Oei, Tao Xie, and Darko Marinov. 2019. iFixFlakies: A framework for automatically fixing order-dependent flaky tests. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 545–555.

[23] Roberto Verdecchia, Emilio Cruciani, Breno Miranda, and Antonia Bertolino. 2021. Know you neighbor: Fast static prediction of test flakiness. *IEEE Access* 9 (2021), 76119–76134.

[24] Ruixin Wang, Yang Chen, and Wing Lam. 2022. iPFlakies: A framework for detecting and fixing python order-dependent flaky tests. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 120–124.

[25] Anjiang Wei, Pu Yi, Zhengxi Li, Tao Xie, Darko Marinov, and Wing Lam. 2022. Preempting flaky tests via Non-Idempotent-Outcome tests. In *International Conference on Software Engineering*. 1730–1742.

[26] Pu Yi, Anjiang Wei, Wing Lam, Tao Xie, and Darko Marinov. 2021. Finding polluter tests using Java PathFinder. *ACM SIGSOFT Software Engineering Notes* 46, 3 (2021), 37–41.

[27] Peilun Zhang, Yanjie Jiang, Anjiang Wei, Victoria Stodden, Darko Marinov, and August Shi. 2021. Domain-specific fixes for flaky tests with wrong assumptions on underdetermined specifications. In *2021 IEEE/ACM 43rd International Conference on Software Engineering*. IEEE, 50–61.

[28] Celal Ziftci and Diego Cavalcanti. 2020. De-flake your tests: Automatically locating root causes of flaky tests in code at google. In *2020 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 736–745.